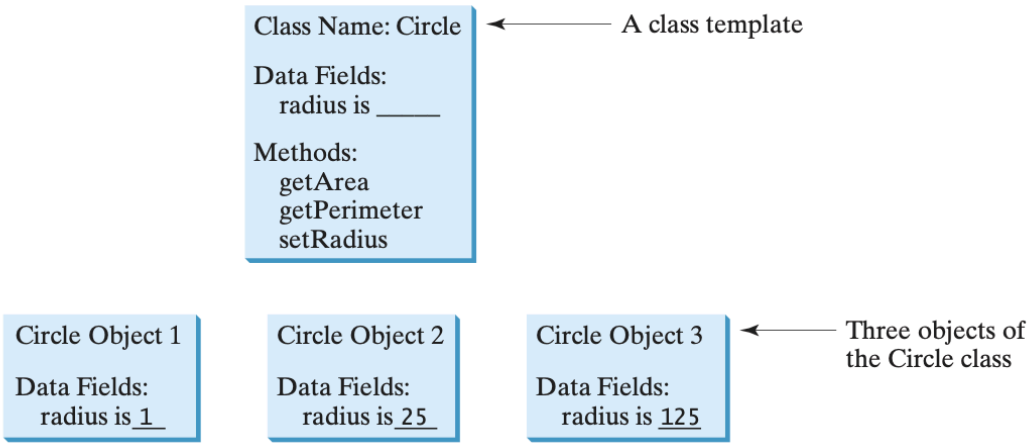
Defining Classes for Objects

In this week's lab, we explore the fundamental concepts of object-oriented programming (OOP), a core approach used in modern software development. OOP allows us to model a program in the same way we understand the real world through objects. An object represents anything that can be clearly identified, such as a student, a desk, a circle, or even a loan. Every object has an identity, a state, and a behavior.

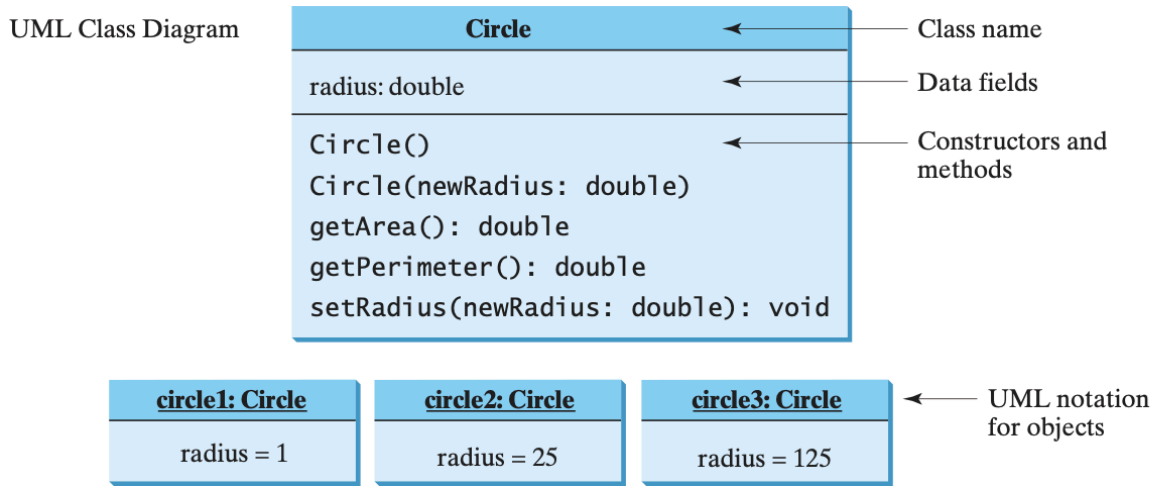
- The **state** of an object is described by its attributes or data fields; for example, a circle has a radius, while a rectangle has width and height. These properties capture the current condition of the object.
- The **behavior** of an object is defined by methods, which describe the actions the object can perform. A circle, for instance, can calculate its area or perimeter through methods like `getArea()` or `getPerimeter()`, and it can update its size using a method like `setRadius()`.



The Circle class is different from all of the other classes you have seen thus far. It does not have a main method and therefore cannot be run; it is merely a definition for circle objects. The class that contains the main method will be referred to in this book, for convenience, as the main class.

The illustration of class templates and objects in above Figure can be standardized using Unified Modeling Language (UML) notation. This notation is called a UML class diagram, or simply a class diagram. In the class diagram, the data field is denoted as

```
dataFieldName: dataFieldType
```




```java
class Circle {
  /** The radius of this circle */
  double radius = 1;

  /** Construct a circle object */
  Circle() {
  }

  /** Construct a circle object */
  Circle(double newRadius) {
    radius = newRadius;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * Math.PI;
  }

  /** Return the perimeter of this circle */
  double getPerimeter() {
    return 2 * radius * Math.PI;
  }

  /** Set new radius for this circle */
  double setRadius(double newRadius) {
    radius = newRadius;
  }
}
```

Data field

Constructors

Method

The program constructs three circle objects with radius 1, 25, and 125 and displays the radius and area of each of the three circles. It then changes the radius of the second object to 100 and displays its new radius and area.

Defining Classes and Creating Objects

Lets see two examples of defining classes and uses the classes to create objects. Listing is a program that defines the Circle class and uses it to create objects. The program constructs three circle objects with radius 1, 25, and 125 and displays the radius and area of each of the three circles. It then changes the radius of the second object to 100 and displays its new radius and area.

To avoid a naming conflict with several enhanced versions of the Circle class introduced later in the chapter, the Circle class in this example is named SimpleCircle. For simplicity, we will still refer to the class in the text as Circle.

```java
// Define the circle class with two constructors
class SimpleCircle {
  double radius;

  /** Construct a circle with radius 1 */
  SimpleCircle() {
    radius = 1;
  }

  /** Construct a circle with a specified radius */
  SimpleCircle(double newRadius) {
    radius = newRadius;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * Math.PI;
  }

  /** Return the perimeter of this circle */
  double getPerimeter() {
    return 2 * radius * Math.PI;
  }

  /** Set a new radius for this circle */
  void setRadius(double newRadius) {
    radius = newRadius;
  }
}
```

```java
public class TestSimpleCircle {
  /** Main method */
  public static void main(String[] args) {
    // Create a circle with radius 1
    SimpleCircle circle1 = new SimpleCircle();
    System.out.println("The area of the circle of radius "
      + circle1.radius + " is " + circle1.getArea());

    // Create a circle with radius 25
    SimpleCircle circle2 = new SimpleCircle(25);
    System.out.println("The area of the circle of radius "
      + circle2.radius + " is " + circle2.getArea());

    // Create a circle with radius 125
    SimpleCircle circle3 = new SimpleCircle(125);
    System.out.println("The area of the circle of radius "
      + circle3.radius + " is " + circle3.getArea());

    // Modify circle radius
    circle2.radius = 100; // or circle2.setRadius(100)
    System.out.println("The area of the circle of radius "
      + circle2.radius + " is " + circle2.getArea());

  }
}
```

Output:

```
The area of the circle of radius 1.0 is 3.141592653589793
The area of the circle of radius 25.0 is 1963.4954084936207
The area of the circle of radius 125.0 is 49087.385212340516
The area of the circle of radius 100.0 is 31415.926535897932
```

The program contains two classes. The first of these, **TestSimpleCircle**, is the main class. Its sole purpose is to test the second class, **SimpleCircle**. Such a program that uses the class is often referred to as a client of the class. When we run the program, the Java runtime system invokes the main method in the main class.